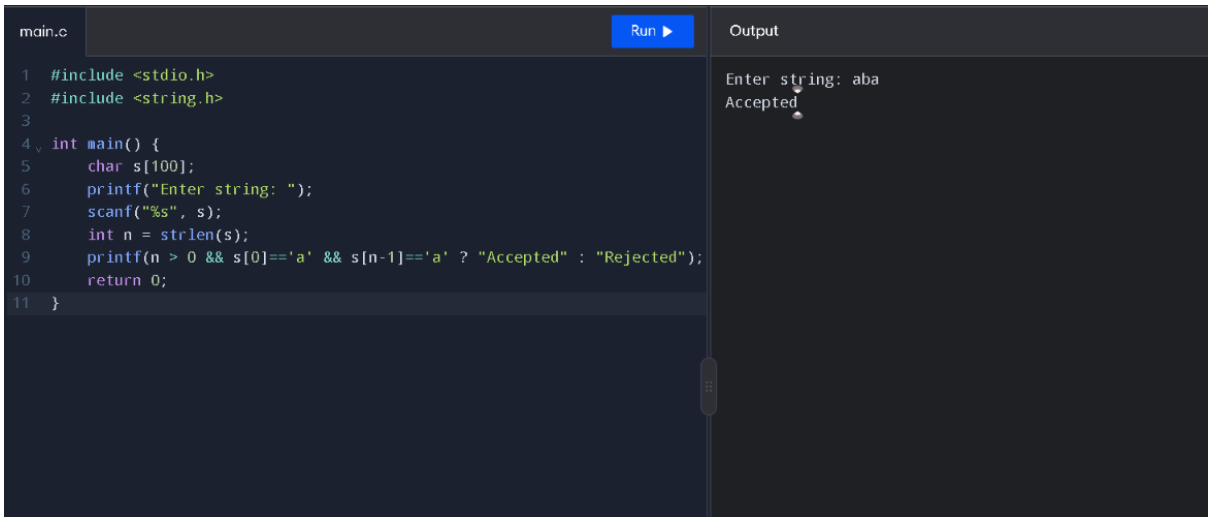
TOC LAB EXPERIMENTS(1-6)

Exp 1:

Aim: Write a C program to simulate a Deterministic Finite Automata (DFA) for the given language representing strings that start with a and end with a

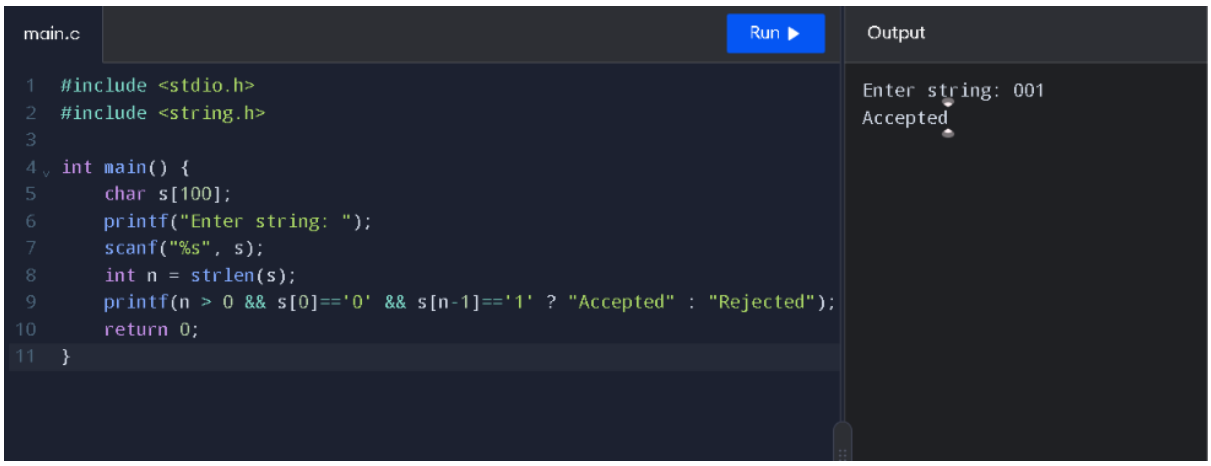


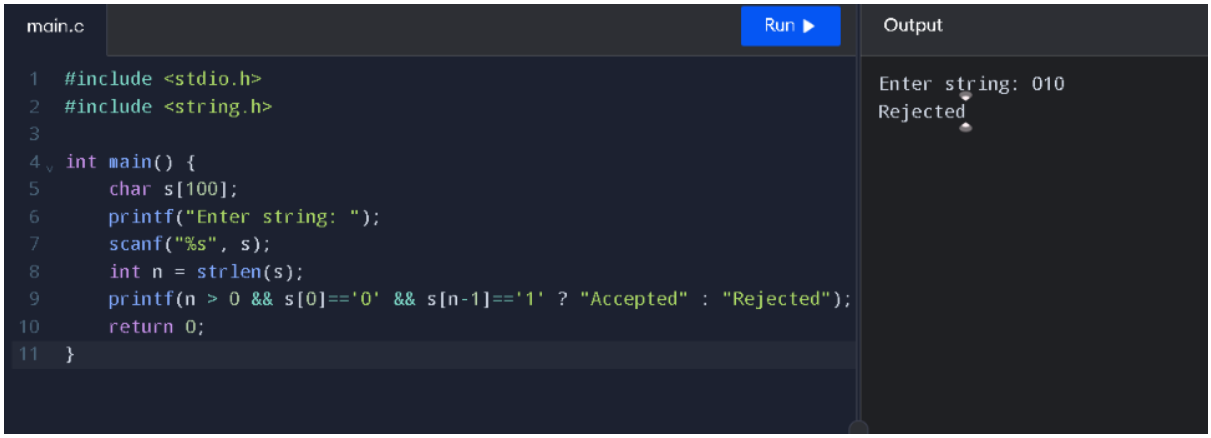


Result: The program accepts strings that start with a and end with a; otherwise, it rejects them.

Exp 2:

Aim : Write a C program to simulate a Deterministic Finite Automata (DFA) for the given language representing strings that start with 0 and end with 1



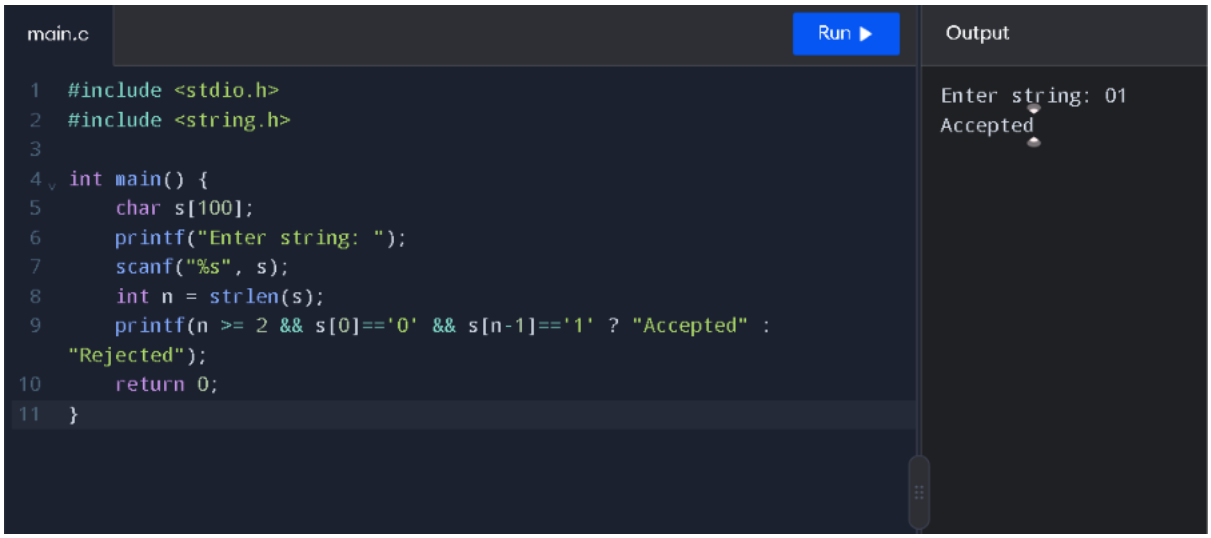


Result: The program accepts strings that start with 0 and end with 1; otherwise, it rejects them.

Exp 3:

Aim: Write a C program to check whether a given string belongs to the language defined by a Context Free Grammar (CFG)

S → 0A1 A → 0A | 1A | ε





Result: The program accepts strings that start with 0 and end with 1; otherwise, it rejects them.

Exp 4:

Aim:Write a C program to check whether a given string belongs to the language defined

by a Context Free Grammar (CFG)

S → 0S0 | 1S1 | 0 | 1 | ε

```c
#include <stdio.h>
#include <string.h>

int main() {
    char str[100];
    int i, j, flag = 1;

    printf("Enter a string: ");
    scanf("%s", str);

    i = 0;
    j = strlen(str) - 1;

    while (i < j) {
        if (str[i] != str[j]) {
            flag = 0;
            break;
        }
        i++;
        j--;
    }

    if (flag)
        printf("String belongs to the CFG");
    else
        printf("String does not belong to the CFG");

    return 0;
}
```

```
Enter a string: 010
String belongs to the CFG
```

Result: Thus, the C program to check whether a given string belongs to the language defined by the CFG S → 0S0 | 1S1 | 0 | 1 | ε was executed successfully.

Exp 5:

Aim: Write a C program to check whether a given string belongs to the language defined

by a Context Free Grammar (CFG)

S → 0S0 | A A → 1A | ε

```c
#include <stdio.h>
#include <string.h>

int main() {
    char str[100];
    int i = 0, j;

    printf("Enter a string: ");
    scanf("%s", str);

    j = strlen(str);

    while (i < j / 2 && str[i] == '0')
        i++;

    while (i < j && str[i] == '1')
        i++;

    while (i < j && str[i] == '0')
        i++;

    if (i == j)
        printf("String belongs to the CFG");
    else
        printf("String does not belong to the CFG");

    return 0;
}
```

```
Enter a string: 010
String belongs to the CFG
```

Result: Thus, the C program to check whether a given string belongs to the language defined by the CFG S → 0S0 | A, A → 1A | ε was executed successfully.

Exp 6:

Aim: Write a C program to check whether a given string belongs to the language defined

by a Context Free Grammar (CFG)

S → 0S1 | ε

```c
#include <stdio.h>
#include <string.h>

int main() {
    char str[100];
    int i, n, flag = 1;

    printf("Enter a string: ");
    scanf("%s", str);

    n = strlen(str);

    if (n % 2 != 0)
        flag = 0;
    else {
        for (i = 0; i < n / 2; i++) {
            if (str[i] != '0' || str[n - 1 - i] != '1') {
                flag = 0;
                break;
            }
        }
    }

    if (flag)
        printf("String belongs to the CFG");
    else
        printf("String does not belong to the CFG");

    return 0;
}
```

```
Enter a string: 0011
String belongs to the CFG
```

Result: Thus, the C program to check whether a given string belongs to the language defined by the CFG S → 0S1 | ε was executed successfully.

Exp 7:

Aim: To write a C program to check whether a given string belongs to the language defined by the CFG.

```c
#include <stdio.h>
#include <string.h>

int main() {
    char s[100];
    int i, found = 0;

    printf("Enter a string: ");
    scanf("%99s", s);

    for (i = 0; s[i] != '\0'; i++) {
        if (strncmp(s + i, "101", 3) == 0) {
            found = 1;
            break;
        }
    }

    if (found)
        printf("String belongs to the CFG");
    else
        printf("String does not belong to the CFG");

    return 0;
}
```

```
Enter a string: 0010
String does not belong to the CFG
```

Result: Thus, the C program to check whether a string belongs to the given CFG was executed successfully.

Exp 8:

Aim: To simulate an NFA that accepts strings beginning with b and ending with a over the alphabet {a,b}

```c
#include <stdio.h>
#include <string.h>

int main() {
    char s[100];
    int n;

    printf("Enter a string: ");
    scanf("%99s", s);

    n = strlen(s);

    if (n >= 2 && s[0] == 'b' && s[n-1] == 'a')
        printf("String accepted");
    else
        printf("String rejected");

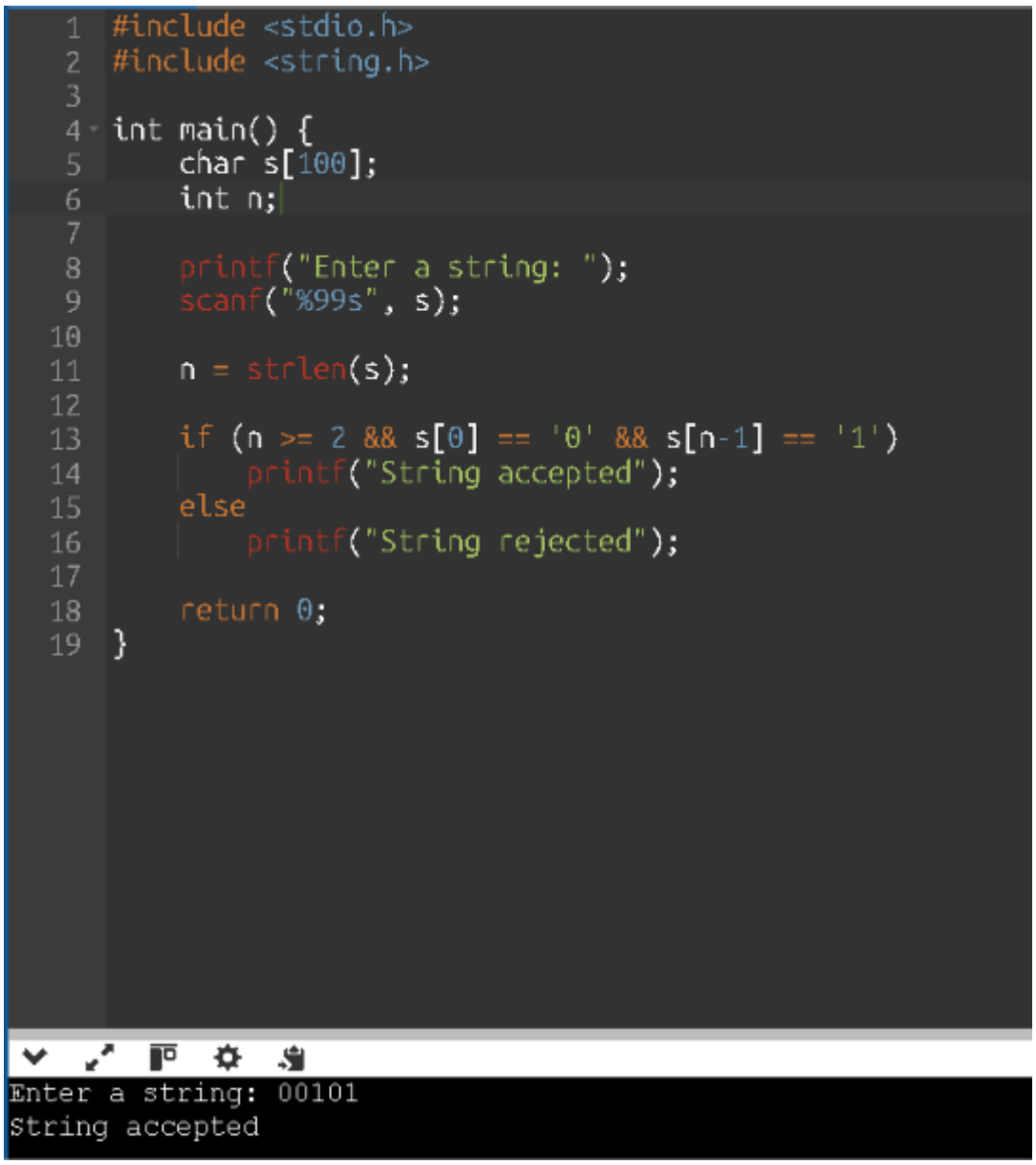
    return 0;
}
```

```
Enter a string: bba
String accepted
```

Result: Thus, the NFA simulation was implemented successfully. For example, bba is accepted.

Exp 9:

Aim: To simulate an NFA that accepts strings beginning with o and ending with 1.



```c
#include <stdio.h>
#include <string.h>

int main() {
    char s[100];
    int n;

    printf("Enter a string: ");
    scanf("%99s", s);

    n = strlen(s);

    if (n >= 2 && s[0] == '0' && s[n-1] == '1')
        printf("String accepted");
    else
        printf("String rejected");

    return 0;
}
```

```
Enter a string: 00101
String accepted
```

Result: Thus, the NFA simulation was implemented successfully. For example, 001 is accepted.

Exp 10:

Aim: To write a C program to calculate the ε-closure of every state in an NFA containing ε-transitions.

```c
#include <stdio.h>

int main() {
    int e[3][3] = {
        {0, 1, 0},
        {0, 0, 1},
        {0, 0, 0}
    };
    int i, j, k, c[3][3] = {0};

    for (i = 0; i < 3; i++)
        c[i][i] = 1;

    for (k = 0; k < 3; k++)
        for (i = 0; i < 3; i++)
            for (j = 0; j < 3; j++)
                c[i][j] |= c[i][k] & e[k][j];

    for (i = 0; i < 3; i++) {
        printf("E-closure(q%d) = {", i);
        for (j = 0; j < 3; j++)
            if (c[i][j]) printf(" q%d", j);
        printf(" }\n");
    }

    return 0;
}
```

```
E-closure(q0) = { q0 q1 q2 }
E-closure(q1) = { q1 q2 }
E-closure(q2) = { q2 }
```

Result: Thus, the ε-closure of each state was calculated successfully.